INDUSTRIAL STRENGTH SOFTWARE SERVICES
TECHNICAL REPORT · ISSS-TR-0505

READING DATABASES

DYNAMODB AND LMDB, READ CLOSELY FOR A BITCASK AND A BOOK ABOUT DATABASES

AUTHOR
Javier Fernández

CONTACT
jfernand@me.com

ISSUED
2026-11-12

ABSTRACT

Append Only has reached the edge of its first part. Chapter 3 turns a query like `SELECT * WHERE key BETWEEN 'a' AND 'c'` into a plan over a sorted keydir, and the next step, in every textbook, is tables with many columns: DDL, a catalog, migrations, and a secondary index per question anyone might ask. This report reads the 2022 USENIX ATC paper on Amazon DynamoDB closely, section by section, with its predecessor Dynamo (2007), its transactions paper (2023), and the data-model sections of Google's Spanner (2012) as companions. It finds that the paper says almost nothing about the data model, and that the little it says is decisive: only the primary key has a schema; the key is hashed on its first part and sorted on its second; and SimpleDB, the service DynamoDB replaced, failed partly because it indexed every attribute. Spanner arrives at the same place from the relational side: rows must have names, and the names decide locality. The report ends with a proposal for the book's next chapters that follows those papers instead of the textbook: schema at open time, queries as a tree, composite keys, one keyspace for several kinds of record, few and explicit indices, and the log as the source of truth. Part II reads the opposite extreme, LMDB: one file, one writer, a copy-on-write B+tree in a memory map, no cache and no log. It reads the 2011 paper that introduced it with the material on lmdb.tech, tests its claims against two independent studies (it is nearly crash-proof, and mmap is right for the workload it chose), and draws what it means for yabiir: snapshots, the batch record as a transaction, measuring merge, and keeping `pread`.

DOCUMENT CONTROL

PART I	Elhemali et al., Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service, USENIX ATC 2022, pp. 1037–1048
PART II	Chu, MDB: A Memory-Mapped Database and Backend for OpenLDAP, LDAPCon 2011, with lmdb.tech; tested against Pillai et al. (OSDI 2014) and Crotty et al. (CIDR 2022)
COMPANIONS	Dynamo (SOSP 2007) · DynamoDB transactions (ATC 2023) · Spanner (OSDI 2012, §2 and §5 only) · Bitcask (Basho 2010)
PURPOSE	Decide what Append Only builds after Chapter 3, and what it can skip

COPIES	`refs/` — every paper cited here, as PDF
RELATED	Append Only chapters 2–4 · ISSS-TR-0503 (email-mcp on yabiir and a table layer)

PART I

DYNAMODB

A managed key-value service at 89 million requests a second: the 2022 paper, its predecessor, its transactions, and Spanner for contrast.

01 WHY READ THIS PAPER NOW

Chapter 3 of Append Only ends with a working pipeline: text, tree, plan, rows. The tree has three fields and two kinds of condition; the planner chooses between `fold`, `get` and `scan`; the executor pushes rows through `Filter`, `Project` and `Distinct`. Everything after that in a database textbook comes as one package. A table has many columns, so there is a `CREATE TABLE`, a catalog that stores it, an `ALTER TABLE` that changes it and a migration story for the rows already written. Queries can filter on any column, so there are secondary indices, and once there are indices the planner has to choose among them, so it needs statistics and a cost model.

That road is long and most of it is machinery. The question behind this report is whether there is a shorter road that is still honest: one that ends at a store real applications could use, without taking the detour through general-purpose DDL. The plan on the table is:

- **the schema is given when the store is opened**, as code, not stored as data and changed by statements;
- **queries are built as trees**, using Chapter 3's AST directly; SQL text stays a front end;
- **migrations are out of scope**.

That leaves the question of what comes next, and DynamoDB is the obvious system to ask. It serves, by the paper's account, 89.2 million requests per second at its 2021 peak [1], and its interface is narrower than any SQL database's: a key schema fixed at creation, a handful of item operations, and indices you add one at a time. If that interface carries Amazon's retail site, then a book can build up to something like it and stop there. Its readers will have built a real database, even without the textbook's whole package.

The paper turns out to be mostly about something else: running a multi-tenant service, with admission control, durability, availability and deployment. Most of it is irrelevant to a single-node store, and some of it is the most useful systems writing Amazon has published. This report reads all of it, because both halves matter, and marks throughout what applies to yabiir.

> HOW TO READ THIS REPORT
> §2–§9 follow the paper's own order. §2 is history, §3 the data model, §4 the architecture, §5 capacity, §6 durability, §7 availability and §8 the benchmarks. §9 is what the paper leaves out, filled in from the transactions paper and the documentation. §10 reads Spanner's data model for contrast. §11 draws the common thread. §12 is how applications are designed for DynamoDB, §13 how it reaches 89 million requests a second, and §14 the proposal for the book. Each section of the reading ends with a short For yabiir note. Quotations are from the paper unless another source is cited. Part II, LMDB, starts at §16 and has its own guide.

02 WHERE DYNAMODB COMES FROM

The paper's §2 is short and is the part to read most carefully. It names three things Amazon had before DynamoDB, and what was wrong with each.

Relational databases, used directly. “Providing applications with direct access to traditional enterprise database instances led to scaling bottlenecks such as connection management, interference between concurrent workloads, and operational problems with tasks such as schema upgrades.” The response was organisational before it was technical: every application’s data went behind a service API, so the database underneath could change without the clients knowing. Note the phrase schema upgrades. It is the first item on the list of operational problems, and it is the problem this report’s plan proposes to avoid.

Dynamo. The 2007 paper describes a store built for the shopping cart, which had to accept writes “even if disks are failing, network routes are flapping, or data centers are being destroyed by tornados” [2]. Its query model was a single sentence: “simple read and write operations to a data item that is uniquely identified by a key,” with “no operations span multiple data items and there is no need for relational schema.” It got its availability from consistent hashing, vector clocks, sloppy quorums and hinted handoff, and paid for it with eventual consistency and conflicts that applications had to merge. Its tradeoff was the right one for the cart. But “Dynamo was a single-tenant system and teams were responsible for managing their own Dynamo installations,” and “the resulting operational complexity became a barrier to adoption.” Engineers chose the managed services instead, “even though the functionality of Dynamo was often better aligned with their applications’ needs.”

SimpleDB. Amazon’s first managed NoSQL service had a table interface and a restricted query set, and two limits. A table held at most 10 GB, so large applications had to split their data across tables by hand. And writes were slow and unpredictable, “which stemmed from the fact that all table attributes were indexed, and the index needed to be updated with every write.”

> THE SENTENCE THIS REPORT IS ABOUT
> “Unpredictable query and write latencies, which stemmed from the fact that all table attributes were indexed, and the index needed to be updated with every write.” This is the paper’s only direct statement about indexing policy, and it is an argument from experience, not taste: Amazon built the index-everything design, ran it as a service, and replaced it. Every index is one more write for every write to the table. When every attribute is indexed, the cost of a write depends on the item’s shape, not on the request, and a service promising single-digit milliseconds can’t make that promise.

DynamoDB, launched in 2012, took “the best parts of the original Dynamo design (incremental scalability and predictable high performance) with the best parts of SimpleDB (ease of administration of a cloud service, consistency, and a table-based data model that is richer than a pure key-value store).” It shared “most of the name of the previous Dynamo system but little of its architecture”. The transactions paper repeats the warning not to confuse the two [3]. DynamoDB has strongly consistent reads, Paxos rather than sloppy quorums, and no vector clocks.

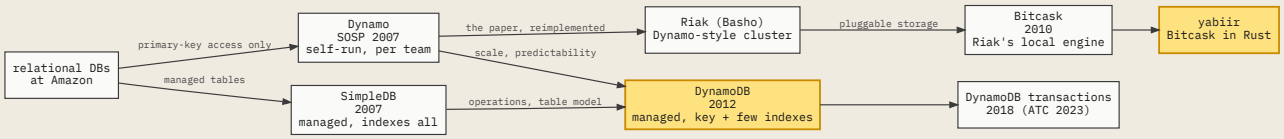


Figure 1: Two branches from the Dynamo paper. Amazon’s managed service took Dynamo’s scaling and SimpleDB’s operations. Basho’s Riak reimplemented the paper, and Bitcask was written as its pluggable local storage engine [4], which yabiir in turn implements.

The lower branch of Figure 1 matters for this book. Dynamo’s own nodes used “pluggable” local persistence. Most production instances ran Berkeley DB, and the engine was chosen by object size [2]. Riak, Basho’s open-source reimplementation of the Dynamo design, kept that plug, and the Bitcask paper opens by saying that “the origin of Bitcask is tied to the history of the Riak distributed database. In a Riak key/value cluster, each node uses pluggable local storage” [4]. yabiir is therefore a descendant of the 2007 paper. It isn’t the distributed part: it’s the engine that sits under one node. A book built on yabiir is reading DynamoDB’s family history from the bottom up.

> FOR YABIIR
> Two lessons come from §2 alone. Schema upgrades were an operational problem large enough to reshape Amazon’s architecture, so treating them as out of scope is a defensible design choice, not a shortcut. And indexing everything is a design Amazon tried and replaced. The book can cite both.

03 THE DATA MODEL: ONE PARAGRAPH

The whole data model is the first two paragraphs of the paper's §3. Here they are, decomposed.

Table, item, attribute "A DynamoDB table is a collection of items, and each item is a collection of attributes." Nothing says what attributes an item has. The introduction is explicit: "DynamoDB tables don't have a fixed schema but instead allow each data item to contain any number of attributes with varying types, including multi-valued attributes."

Primary key "Each item is uniquely identified by a primary key. The schema of the primary key is specified at the table creation time." This is the only schema there is.

Partition key, sort key "The primary key schema contains a partition key or a partition and sort key (a composite primary key). The partition key's value is always used as an input to an internal hash function. The output from the hash function and the sort key value (if present) determines where the item will be stored. Multiple items can have the same partition key value in a table with a composite primary key. However, those items must have different sort key values."

Secondary indices "A table can have one or more secondary indexes. A secondary index allows querying the data in the table using an alternate key, in addition to queries against the primary key."

Operations Table 1 of the paper lists four: `PutItem`, `UpdateItem`, `DeleteItem`, `GetItem`. "Any operation that inserts, updates, or deletes an item can be specified with a condition that must be satisfied in order for the operation to succeed." ACID transactions are mentioned in one sentence.

Read slowly, the partition-key sentence is the most important in the paper for this book. The store's order is not the key's order. Items are placed by the pair (hash of partition key, sort key). Two items with different partition keys land wherever the hash puts them, with no relation between their positions. Items that share a partition key sit together, sorted by sort key. A composite primary key is therefore **a hash index on its first part and a sorted index on its second**: exactly the two structures of Append Only's chapters 1 and 2, used together, each for the job the Chapter 2 footnote says it is built for. The hash spreads load across partitions. The sort order makes ranges cheap within one partition key, and only there.

DYNAMODB	YABIIR TODAY (0.6.0)	WHAT THE BOOK WOULD NEED
item, attributes	one value, opaque bytes	a self-describing value: named attributes
partition key, hashed	the key, in a hashed keydir (`KeyOrder::Hashed`)	the first part of a composite key
sort key, ordered	the key, in a sorted keydir (`KeyOrder::Sorted`)	the rest of the composite key, encoded to sort correctly
`GetItem`	`get`	—
`PutItem`, `DeleteItem`	`put`, `delete`	a condition: "only if absent", "only if present"
`Query`: one partition key, a sort-key range	`scan(start, end)`, forward only	a prefix range, either direction, with a limit
`Scan`	`fold`	—
secondary index	none	another key written with the item, atomically
transactions	none: one record per write	an atomic batch of records

The table's last column is the subject of §14. Two things stand out before going on. yabiir already has both halves of DynamoDB's key, but as a choice made at open time between them rather than one composite key. And the gap between yabiir and DynamoDB's data model is three pieces of engineering, not a new architecture: composite keys, self-describing values and atomic batches.

WHAT §3 DOESN'T SAY

The paper never describes `Query`, the operation that reads a range of sort keys within one partition key. It is the operation that makes the composite key useful. It never says what a secondary index costs, how many a table can have, or whether they are consistent with the table. Those facts are in the documentation, and §9 fills them in.

04 ARCHITECTURE

The rest of §3 is the physical design. A table is split into **partitions**, each "a disjoint and contiguous part of the table's key-range". Given the partition-key sentence above, contiguous is in the space of (hash, sort key), not of the keys as written. Each partition is stored as a **replication group** of replicas in different Availability Zones, which use Multi-Paxos for leader election and consensus.

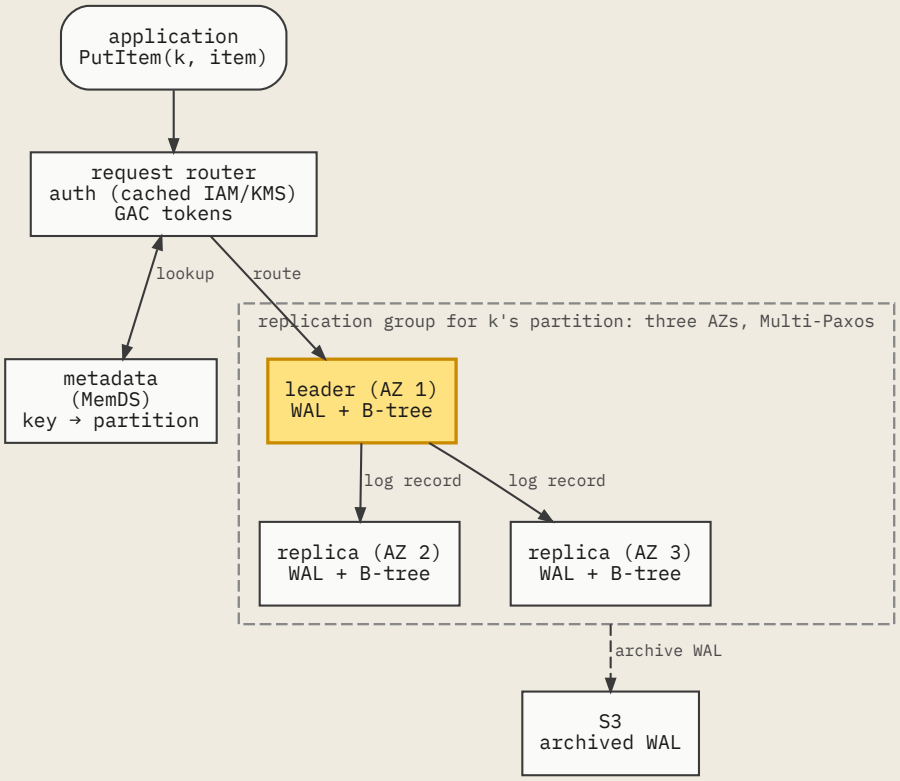


Figure 2: A write, as §3 describes it. The router authenticates the request, looks up the partition that holds the key, and sends the request to that partition's leader. The leader appends a write-ahead log record and sends it to its peers, and acknowledges once a quorum, two of the three, has persisted it. When a replica fails, a log replica that holds only recent log records takes its place within seconds (§6).

The write path in Figure 2 has four rules, all stated in the paper:

1. **Only the leader writes.** "Only the leader replica can serve write and strongly consistent read requests." Any replica serves eventually consistent reads.
2. **A write is durable at a quorum.** The leader generates a write-ahead log record and sends it to its peers. "A write is acknowledged to the application once a quorum of peers persists the log record to their local write-ahead logs."
3. **Leadership is a lease.** A leader keeps its role by renewing a lease, and "the new leader won't serve any writes or consistent reads until the previous leader's lease expires". Without that rule, two leaders could both believe they were current.
4. **Storage is a log plus a B-tree.** "A replication group consists of storage replicas that contain both the write-ahead logs and the B-tree that stores the key-value data." **Log replicas** hold only recent log entries. They are "akin to acceptors in Paxos" and exist for availability and durability (§6, §7).

Around the storage nodes are tens of microservices, of which the paper names four. The **metadata service** maps keys to partitions and partitions to nodes. **Request routers** authenticate and route every request. The

storage service is the fleet of nodes, each hosting "many replicas of different partitions". The **autoadmin service** is "the central nervous system": it handles fleet health, partition health, scaling, and every control-plane request (creating tables, changing indices). Point-in-time restore, backups, streams, global admission control, global tables, global secondary indices and transactions are separate services that the figure leaves out.

LOG AND B-TREE, OR LOG AS THE STORE

The storage replica is the classic database design: a write-ahead log for durability and a B-tree as the real store, with the log truncated once its records are in the B-tree and archived. Bitcask inverts it. The log is the store, and the index (the keydir) lives in memory, rebuilt from the log or from hint files on open. yabiir's sorted keydir is a B-tree too, a `BTreeMap`, but in memory, over a log that is never rewritten except by merge. The difference shows up throughout §6: DynamoDB has to archive its log to S3 to keep history, while a Bitcask keeps history until merge throws it away.

FOR YABIIR

None of the distributed machinery applies to a single node. The rule that does apply is the shape of the write path: **a write is done when its log record is durable**, and every other structure is derived from the log. That is already yabiir's design, and `sync_on_put` with group commit (ISSS-TR-0502) is its single-node version of "a quorum has persisted the record."

05 FROM PROVISIONED TO ON-DEMAND: A LESSON ABOUT SKEW

§4 is the longest section of the paper and is about one mistake, made at launch and corrected over a decade. The mistake was assuming load is uniform.

5.1 THE PROVISIONED MODEL

At launch, customers bought throughput in **read capacity units** and **write capacity units**. One RCU is one strongly consistent read per second of an item up to 4 KB, and one WCU is one write per second of an item up to 1 KB. DynamoDB divided a table's throughput equally among its partitions, and each storage node admitted requests against its local partitions' allocations. A partition had a maximum (1,000 WCUs in the paper's example), so a table provisioned for 3,200 WCUs got four partitions of 800 each. Raised to 3,600, each partition got 900. Raised to 6,000, the four split into eight of 750.[1]

5.2 TWO FAILURES

The equal split "is based on the assumptions that an application uniformly accesses keys in a table and the splitting a partition for size equally splits the performance." Neither held.

Hot partitions traffic concentrated on a few items. Those items' partition ran out of its share while the table as a whole had capacity to spare, and requests were **throttled**, which the application experiences as unavailability, "even though the service was behaving as expected."

Throughput dilution when a partition split because it held too much data, its throughput was divided between the children. If the hot keys all landed in one child, that child had half the throughput the hot keys had before the split.

Customers responded by over-provisioning: buying capacity they didn't use, so that the hot partition's share would be enough. That worked, and it was, in the paper's words, "a poor experience because it was difficult to estimate the right level of performance provisioning."

[1] The paper continues: "If the table's capacity was decreased to 5000 WCUs, then each partition's capacity would be decreased to 675 WCUs." Eight partitions sharing 5,000 WCUs get 625 each. Either the example is wrong or something unstated changed the partition count; the arithmetic suggests a typo.

5.3 FIVE FIXES, EACH LESS STATIC THAN THE LAST

FIX	MECHANISM	LIMIT THE PAPER NAMES
Bursting	Keep up to 300 s of a partition's unused capacity and spend it in spikes, if the node has spare tokens. Two token buckets per partition (allocated, burst) and one per node. Burst writes also check the other replicas' node buckets.	Short spikes only, and only on a node with headroom.
Adaptive capacity	Watch for throttling on tables under their total, and boost the hot partitions' allocations with a proportional controller. Move boosted partitions to nodes that can serve them.	Reactive: the customer has already been throttled. Yet it "eliminated over 99.99% of the throttling due to skewed access pattern."
Global admission control	Take admission away from partitions. Routers hold local, time-limited tokens and refill them from GAC servers, which track each table's total consumption. Partition buckets stay, capped, "for defense-in-depth."	Partitions can now always burst, so nodes can be overcommitted.
Balancing	Nodes are packed beyond their nominal capacity, so each one watches its own throughput and size, and over a threshold asks autoadmin to move replicas elsewhere.	—
Split for consumption	Split a partition whose consumed throughput crosses a threshold, at a point chosen from "the key distribution the partition has observed," not the middle.	Useless for a single hot item, or a key range "accessed sequentially"; both are detected and not split.

The last row of the table matters for this book, and §14 returns to it. A workload that reads a key range in order can't be spread by splitting the range, because the reader is always at one end of it. A key that's a timestamp, with writes always at the newest end, has the same problem for writes. DynamoDB's answer is the hash on the partition key, and its advice to customers is to choose partition keys with many distinct, evenly used values. The paper doesn't say it here, but the design forces it: **the key schema is where an application decides its own skew.**

5.4 ON-DEMAND

The end state is **on-demand** tables, which have no provisioning at all. DynamoDB watches consumed reads and writes and "instantly accommodates up to double the previous peak traffic on the table"; beyond that it adds capacity by splitting for consumption. The customer no longer predicts load. The system predicts it, from the table's own history.

> FOR YABIIR
> Admission control has no single-node analogue worth building. Two lessons carry over. First, **don't assume uniform access**, especially in benchmarks: yabiir's `loadtest` picks keys uniformly from its keyspace, which is the friendliest possible workload. A skewed distribution (Zipfian, the YCSB default) would be a better default for the measurements the book prints. Second, the observation that sequential key access defeats range splitting is the same observation that will make the email program's "newest twenty" a reverse range scan, which yabiir doesn't yet have.

06 DURABILITY AND CORRECTNESS

§5 opens with “Data should never be lost after it has been committed,” and then treats data loss as something to prevent, detect and correct, in that order, against three causes: hardware failures, silent data errors, and software bugs.

6.1 HARDWARE FAILURES: THE LOG, TWICE

Write-ahead logs are on all three replicas and are “periodically archived to S3.” Unarchived log is “typically a few hundred megabytes” per replica. When a node fails, every replication group on it drops to two copies. Rebuilding a storage replica takes minutes, because the B-tree has to be copied. So the leader first adds a **log replica**, which needs only the recent log and is ready in seconds. Durability is restored fast with a cheap replica, and the expensive one follows.

6.2 SILENT ERRORS: CHECKSUMS EVERYWHERE, AND HOLES

Hardware can store the wrong bytes without failing. DynamoDB keeps “checksums within every log entry, message, and log file” and checks them on every transfer between nodes, “because these messages can go through various layers of transformations before they reach their destination.” Before a log file is archived, the archiving agent checks every entry’s table and partition, every checksum, and “that the log file doesn’t have any holes in the sequence numbers.” Archival agents run on all three replicas. One that finds a file already archived downloads it and compares it with its own log, and S3 checks a content checksum on upload.

6.3 CONTINUOUS VERIFICATION

The most interesting mechanism in the section is the **scrub**. It checks two things: that the three replicas hold the same data, and that “the data of the live replicas matches with a copy of a replica built offline using the archived write-ahead log entries.” The live state is compared against a reconstruction from the history. The paper’s conclusion is strong: “continuous verification of data-at-rest is the most reliable method of protecting against hardware failures, silent data corruption, and even software bugs.” The last clause is the important one. A replica built from the log by different code is a test of the code that built the live replica.

6.4 SOFTWARE BUGS: TLA+

The replication protocol is specified in TLA+ and model-checked, and every feature that touches it is added to the specification first. The paper cites the CACM article on AWS’s use of formal methods [5]. The control plane and transactions have been verified the same way, and failure injection and stress tests run on every deployment.

6.5 BACKUPS AND POINT-IN-TIME RESTORE

Backups and restores are built from the archived logs, so they “don’t affect performance or availability of the table.” Point-in-time restore covers the previous 35 days. DynamoDB takes periodic snapshots of each partition, timed by how much log has accumulated. To restore, it takes the closest snapshot and applies the log up to the requested timestamp. Backups are consistent across partitions “up to the nearest second.”

FOR YABIIR — THE SECTION THAT MAPS MOST DIRECTLY

yabiir already has the parts DynamoDB built this on. Every data-file entry has a CRC32 over its header and payload. Hint files have a whole-file CRC32, and a bad one falls back to scanning the data file. Opening a store rebuilds the keydir from the log. Three things follow:

- **A scrub is a rebuild and a comparison.** `verify` would open a second, read-only keydir from the data files alone (no hint files) and compare it entry by entry with the live one. It's a small feature with DynamoDB's own argument behind it: it tests the hint-file path and the merge path against the simplest possible reconstruction.
- **Point-in-time reads are almost free in a log**, because every entry already carries a timestamp. A keydir rebuilt from entries with `tstamp ≤ t` is the store as of time t. The catch is merge: it discards superseded entries, which is exactly the history a point-in-time read needs. DynamoDB keeps history by archiving logs before truncating them; a Bitcask would have to keep merged-away files, or merge only before a horizon.
- **yabiir already has a model test**, which runs random sequences of puts, deletes, reopens and merges against a plain `BTreeMap`. That is the lightweight cousin of DynamoDB's TLA+ specification.

07 AVAILABILITY

7.1 THE WRITE QUORUM, AND LOG REPLICAS AGAIN

"A healthy write quorum in the case of DynamoDB consists of two out of the three replicas from different AZs." A partition can take writes while it has a leader and a quorum. When a replica is unresponsive, the leader adds a log replica, which is "the fastest way to ensure that the write quorum of the group is always met." The paper credits the formally verified Paxos implementation with making that change safe to try. DynamoDB runs "millions of Paxos groups in a Region with log replicas."

7.2 GRAY FAILURES

A new leader has to wait for the old leader's lease to expire, a couple of seconds without writes. So a false failure detection costs availability. **Gray failures** [6] are partial: one replica can't reach the leader while the others can, or the routers can't reach it while the replicas can. They caused exactly those false detections. The fix is simple. A follower that wants to start an election first asks the other replicas whether they can reach the leader, and drops the attempt if they can. According to the paper, it "significantly minimized the number of false positives."

7.3 MEASURING IT

Availability is computed per 5-minute interval as the fraction of requests that succeed. It is tracked per table and per service, with alarms that fire when customers see errors ("customer-facing alarms"). It is also measured from outside: by Amazon services that use DynamoDB, and by canary applications that run in every AZ and call every public endpoint.

7.4 DEPLOYMENTS

Two practices here generalise well beyond DynamoDB.

- **Test the rollback.** "It's not just the end state and the start state that matter," because a rollback can leave a state different from the starting one. Every deployment runs upgrade and downgrade tests at the component level, and the software is "rolled back on purpose and tested."
- **Read-write deployments.** A change to a message format or protocol ships in two steps. First deploy code that can read the new format. Once every node runs it, deploy code that writes it. Both formats coexist, and a rollback of the second step is still safe.

Deployments go to a few nodes first, with automatic rollback on alarms. On storage nodes, leaders hand over leadership before they restart, so the new leader doesn't wait for a lease to expire.

7.5 STATIC STABILITY

Every request depends on IAM and, for tables encrypted with customer keys, on KMS. DynamoDB caches their answers in the routers and keeps using them, for a predetermined period, if those services become unavailable. That is **static stability**: when a dependency fails, the system keeps doing what it was doing.

7.6 METADATA AND THE BIMODAL CACHE

At launch, routing metadata lived in DynamoDB itself, and routers cached a whole table's routing on first use. The hit rate was 99.75%. That sounds like success, but the paper calls it bimodal. A router with a cold cache sends every request to the metadata service, and adding new router capacity made metadata traffic spike "up to 75 percent". An ineffective cache becomes a cascading failure, and the paper cites the 2015 us-east disruption.

The fix has two parts. **MemDS** is an in-memory, replicated metadata store built on a "Perkle" tree, a Patricia tree crossed with a Merkle tree. It stores keys in order and supports prefix lookups, ranges, and `floor` / `ceiling` lookups, the operations that find the partition holding a key. And a new router cache **calls MemDS on every hit** as well as every miss, asynchronously. That raises MemDS's load but keeps it constant, whatever the hit rate, so a cold cache changes nothing.

> LESSON FOUR, FROM THE INTRODUCTION
> "Designing systems for predictability over absolute efficiency improves system stability. While components such as caches can improve performance, do not allow them to hide the work that would be performed in their absence, ensuring that the system is always provisioned to handle the unexpected." The constant-work cache is this lesson in its purest form. It does more work than it needs to, on purpose, so that it never has to do a different amount of work.

> FOR YABIIR
> **Read-write deployments are the answer to "punt on migrations,"** applied to file formats rather than messages. If every record and every value carries a format version, a new version of the engine first learns to read the new format, and only a later version writes it. Old files are never rewritten in place. They are rewritten, if at all, by merge, which already copies every live record. That is a migration strategy with no migration step. And note MemDS's `floor` lookup: "which partition holds key k" is "the greatest boundary ≤ k", which a sorted keydir answers and a hashed one can't.

08 MEASUREMENTS

§7 runs YCSB [7] workloads A (50% reads, 50% updates) and B (95% reads, 5% updates) against production DynamoDB in us-east-1, with 900-byte items and a **uniform** key distribution, from 100,000 to 1,000,000 operations per second. Figures 5 and 6 show p50 and p99 latency staying flat across that range, for reads and writes. The claim is narrow and the evidence supports it: latency doesn't change with throughput.

Three things are worth noticing in a close reading:

- **The workload is the friendly one.** §4 is a long account of how non-uniform access broke the original design. The benchmark uses uniform access, which is the case §4 says was never the problem. A Zipfian run, which is YCSB's own default, would have tested the mechanisms §4 describes.
- **There are no absolute numbers in the text**, only the figures, and no comparison with another system. The benchmark shows that latency doesn't depend on scale. It doesn't show how fast DynamoDB is, and doesn't try to.
- **The percentiles are right.** Reporting p99 next to p50 follows the 2007 paper, which measured everything at the 99.9th percentile because "Amazon's engineering and optimization efforts are not focused on averages" [2].

> FOR YABIIR
> `cargo xtask bench` records Criterion's median and mean, with the confidence interval of the median. That suits Criterion's microbenchmarks, which measure one operation in a loop, but it isn't a latency distribution. For the book's claims about `get` versus `scan`, the medians are fine. For any claim about a store under load, `loadtest`'s percentiles are the right numbers, and its key distribution should be skewed.

09 WHAT THE PAPER LEAVES OUT

The paper is about operating a service. For the data model, the most important things are elsewhere.

9.1 QUERY, AND THE SHAPE OF A RANGE

`Query` reads items with one partition key, optionally restricted by a condition on the sort key: `=`, `<`, `<=`, `>`, `>=`, `BETWEEN` or `begins_with`. It returns them in sort-key order, forward or reverse, with a limit and paging [8]. That is the whole range-query interface. There is no range across partition keys: the hash has scattered them. `Scan` reads the whole table. Every efficient read in DynamoDB is therefore a `GetItem` or a `Query`, which is a lookup in the hash index followed by a range in the sorted one.

9.2 SECONDARY INDICES, AND HOW FEW

The documentation describes two kinds of secondary index [8].

- A **local secondary index** keeps the table's partition key and replaces the sort key with another attribute. It has to be created with the table, and a table can have at most five.
- A **global secondary index** has a different partition key and sort key altogether, and is maintained asynchronously. Reads from it are eventually consistent only. The default quota is twenty per table.

An index can copy all of an item's attributes, only the keys, or a chosen list of them, and every write to the table is also a write to every index the item appears in. That's the SimpleDB lesson built into the pricing: indices are charged in write capacity, so their cost shows up on the bill.

There's a third technique that the documentation and DynamoDB's practitioners teach, which needs no index at all. Several kinds of records share one table and one key space, with key prefixes that make related records adjacent. Then one `Query` returns a mailbox and its newest messages, or a customer and their orders. Its usual name is single-table design, and §10 shows that Spanner built the same idea into its schema language.

9.3 TRANSACTIONS

DynamoDB added transactions in 2018. The 2023 paper describes them [3], and they follow the same rule as everything else: **predictable cost per request**.

- **One request, no session.** "Transactions are submitted as single request." There's no `BEGIN` and `COMMIT`, because long-running transactions would hold resources on servers shared by tenants. `TransactWriteItems` and `TransactGetItems` succeed or fail at once, without blocking.
- **No locks.** Concurrency control is optimistic, using **timestamp ordering**. A coordinator gives each transaction a timestamp from its local clock, kept accurate by AWS's time-sync service, and runs a two-phase protocol, prepare then commit, across the items' storage nodes, and each node checks the transaction against its items' timestamps.
- **Single-version.** There's no MVCC, which would have meant "major changes to the storage servers," retention policies and storage costs. The paper accepts that read-only and read-write transactions can therefore conflict.
- **Singleton operations bypass it.** Plain gets and puts go straight to the storage nodes, "while still being serialized with respect to multi-item transactions."
- **Recovery is a table.** Coordinators record each transaction in a **ledger**, itself a DynamoDB table keyed by transaction id, and recovery managers scan it for transactions that stalled.

9.4 OTHERS

Streams and global tables (multi-region replication) are mentioned in passing and never described. Neither is needed here.

> FOR YABIIR
> An index written by the store has to be written atomically with the row it indexes, or it can disagree with the table after a crash. yabiir writes one record per `put`, with no batch, so there's nothing to make two records atomic yet. A **batch record**, several puts and deletes in one CRC-covered entry that recovery applies all or none of, is the one piece of storage-engine work the proposal in §14 needs. DynamoDB's one-request transactions are a model for its API: a list of writes, submitted once, with conditions.

10 SPANNER, FOR THE OTHER SIDE

Spanner came from the opposite direction. Google's Bigtable was a sorted, versioned key-value store, and applications kept asking it for "complex, evolving schemas" and strong consistency across data centres. More than 300 applications chose Megastore, despite its poor write throughput, because its "semi-relational data model" was simpler to manage [9]. Spanner became a SQL database with general transactions. Its data-model section still reads like DynamoDB's.

Rows must have names. "Spanner's data model is not purely relational, in that rows must have names. More precisely, every table is required to have an ordered set of one or more primary-key columns. This requirement is where Spanner still looks like a key-value store: the primary keys form the name for a row." The reason given is the one DynamoDB's sort key implies: "Imposing this structure is useful because it lets applications control data locality through their choices of keys."

Interleaving. Tables are declared in hierarchies. The paper's own example (Figure 4):

SPANNER SCHEMA, FROM FIGURE 4 OF THE PAPER

```
CREATE TABLE Users {
  uid INT64 NOT NULL, email STRING
} PRIMARY KEY (uid), DIRECTORY;

CREATE TABLE Albums {
  uid INT64 NOT NULL, aid INT64 NOT NULL,
  name STRING
} PRIMARY KEY (uid, aid),
  INTERLEAVE IN PARENT Users ON DELETE CASCADE;
```

`Albums(2,1)` is stored right after `Users(2)`, in key order. A user's row together with every row "in descendant tables that start with K in lexicographic order, forms a directory," and the directory is the unit Spanner replicates, places and moves. It is single-table design, declared in the schema rather than in the application's choice of key prefixes. The paper is clear that the system needs it: "Without it, Spanner would not know the most important locality relationships."

Secondary indices were missing, and the application built them. In 2012 Spanner did "not yet provide automatic support for secondary indexes". F1, the advertising backend that was its first customer, "was able to implement their own consistent global indexes using Spanner transactions." In F1's data, most directories (most customers) fit in one fragment, so a customer's reads and writes touch one server. The directories with more than 100 fragments "are all tables that contain F1 secondary indexes". The indices are what spread across machines.

TrueTime, the API that exposes clock uncertainty and makes external consistency possible, is the paper's headline contribution. It doesn't bear on this report's question and isn't covered here.

> FOR YABIIR
> Spanner shows that the conclusion isn't a NoSQL quirk. A system with SQL, a schema language and general transactions still requires every table to have an ordered primary key, still uses the key to decide locality, and started life with no automatic secondary indices. Its first major customer built them as ordinary tables written inside transactions. That is the design §14 proposes for the book.

11 READING ACROSS: THE KEY IS THE INDEX

Put the papers side by side and three claims appear in all of them.

The ordered primary key is the main index DynamoDB stores items by (hash of partition key, sort key). Spanner requires an ordered primary key and stores tables interleaved by it. Bitcask has one index, the keydir, and yabiir 0.6.0 can keep it sorted. In each case, the efficient reads are lookups and ranges over the key, and everything else is a scan.

The schema is only the key DynamoDB fixes the key schema at creation and nothing else. The rest of an item is attributes it carries itself. Spanner has full schemas, but the key is the part that can't change. A DynamoDB table's key schema can't be altered at all; you create a new table. Fixing the key and leaving values self-describing removes the need for migrations without giving up structure where it matters.

Indices are few, explicit, and paid for at write time SimpleDB indexed everything and was replaced. DynamoDB allows at most five local and, by default, twenty global indices, charges for them in write capacity, and keeps the global ones asynchronous. Spanner shipped without automatic indices, and its first customer built its own. In every case, an index exists because someone named a query it serves, and pays for it on every write.

The 700 indices that a general-purpose relational design invites come from a different premise: that queries are unknown when the data is designed, so any column might be filtered on, so it might need an index. DynamoDB's premise is the opposite, and its modelling guidance says so first: **know the access patterns, then design the keys to serve them** [8]. That's a narrower database, and it's the one that serves 89 million requests a second.

12 DESIGNING FOR DYNAMODB: WHAT KEYS ARE FOR

The paper is silent on how to model an application. The documentation [8] and practitioners such as DeBrie [10] are not, and what they teach follows directly from §3's paragraph. This section describes that practice, because it shows the data model in use, and because the book's proposed line (§14) is that practice, done on one node.

12.1 START FROM THE QUESTIONS, NOT THE ENTITIES

A relational design starts from the entities and their relationships, normalises them, and answers questions later with joins and indices. A DynamoDB design runs the other way. **First list every access pattern the application has**: "open a mailbox and show the newest twenty messages", "fetch message 16,720", "every message from this sender". Then design keys so that each pattern is one `GetItem` or one `Query`. The key schema can't be changed after the table is created, so the list has to come first. A pattern nobody listed may have no efficient answer later, short of a new index and a backfill.

That is the price, stated plainly. A relational schema keeps every future question possible, at the cost of not knowing what any of them will cost. A DynamoDB schema makes the listed questions cheap and fixed-cost, and makes unlisted ones expensive.

12.2 THE PARTITION KEY: WHAT YOU ALWAYS KNOW, AND HOW LOAD SPREADS

The partition key is the part of the key that is hashed. Two things follow from that, and they decide what to use it for.

- **Every efficient read must know it exactly.** `GetItem` needs the whole key, and `Query` needs the partition key with `=`. So the partition key is the value the application always has in hand when it asks: the user's id, the account, the order number, the device. If the question is "every order placed today", then today can't be a partition key component unless every query names a day.
- **It decides how load spreads.** The hash places different partition-key values on unrelated partitions, and a partition has a ceiling: the documentation gives 3,000 read units and 1,000 write units per second [8]. One partition key value can never go faster than one partition. A good partition key has **many distinct values, used about evenly**. User and device ids are good. Status (`OPEN`, `CLOSED`), a boolean, a country

or today's date are bad, because a handful of values carry all the traffic: §4's hot partitions, chosen in advance by the schema.

When one value is unavoidably hot (a celebrity's feed, a global counter, "today" in a time series), the documented remedy is **write sharding**. The application appends a suffix from a small range, say `#0` to `#9`, so writes spread over ten partition keys, and reads query all ten and merge the results. The hash can't spread one key, so the application turns it into ten.

12.3 THE SORT KEY: WHAT YOU RANGE OVER, AND IN WHAT ORDER

The sort key is the ordered part. It's for everything a `Query` does after fixing the partition: ranges, prefixes, order and limits. Four uses come up again and again.

Order a timestamp or a monotonic id as the sort key makes "newest n" a reverse `Query` with a limit. It reads n items however many the partition holds.

Hierarchy a sort key spelled from general to specific, such as `USA#CA#San Francisco`, answers every level of the hierarchy with `begins_with`: one country, one state or one city.

Kinds of item a prefix says what the item is: `META`, `MSG#…`, `MEMBER#…`. One `Query` can then return one kind (`begins_with MSG#`) or the parent and its children together (the whole partition).

Versions writing a new item per version (`v1`, `v2`, …) next to a `v0` item that holds a copy of the latest keeps history and makes the current version one `GetItem`.

All of this depends on the sort key's **byte order** being the order the application means. Numbers are zero-padded or encoded big-endian, dates are written ISO 8601, and separators are chosen to sort before every character a component can contain. It's the same problem §14's Chapter 5 solves with the tuple encoding, solved by hand in strings.

12.4 ITEM COLLECTIONS: THE JOIN THAT'S ALREADY DONE

All items that share a partition key form an **item collection**, stored together and sorted by sort key. If an application puts a parent and its children in one collection, with the parent's id as partition key and the children distinguished by sort key, then "the parent and its children" is one `Query`. No join is needed, because the data was stored joined.

Taken to its conclusion, this is **single-table design**. Every entity in the application lives in one table, with generic key attributes (`PK`, `SK`) whose values carry type prefixes. The table stops looking like a relation. It looks like a sorted map of every answer the application needs, which is what it is.

12.5 SECONDARY INDICES: ANOTHER KEY FOR THE SAME ITEMS

A global secondary index re-keys the items for a pattern the primary key can't serve. Practice has turned that one mechanism into several techniques:

- **Re-keying.** "Messages by sender": the index's partition key is the sender, its sort key the date.
- **Sparse indices.** An item appears in an index only if it has the index's key attributes. Write an `unread` attribute only on unread messages, and an index on it holds the unread messages and nothing else. It's a filtered list that costs nothing for the items it excludes.
- **Overloading.** Name an index's attributes generically (`GSI1PK`, `GSI1SK`) and fill them differently for each kind of item. One index then serves a different pattern for each type, which keeps the count well under the quota.
- **The inverted index.** An index whose partition key is the table's sort key, and the other way round, gives a many-to-many relationship from both sides: members of a group, and groups of a member.

Two caveats are part of the design, not footnotes. Global indices are **eventually consistent**: a read just after a write can miss it. And each one is written on every write that touches its attributes, which is SimpleDB's lesson, paid in write units.

12.6 WHAT NOT TO DO

- **Don't answer a pattern with `Scan`.** It reads the whole table, and its cost grows with the table.

- **Don't count on filter expressions.** A `Query` with a filter reads, and is charged for, every item in the key range before the filter drops any. A filter trims what is returned, not what is read. If the filter matters for performance, it belongs in the key.
- **Don't keep large values in items.** Items are limited to 400 KB, and size is what capacity units charge for. Large content goes to S3, with a pointer in the item.
- **Don't compute aggregates at read time.** Counts and totals are kept as attributes, updated with each write (`UpdateItem` with `ADD`), or maintained from the table's stream.
- **Don't rely on an index for uniqueness.** Only the primary key is unique. A second unique field, such as an email address, gets its own item (`PK = EMAIL#…`) written in the same transaction as the user, with the condition that it doesn't exist yet.

12.7 A WORKED EXAMPLE: THE EMAIL PROGRAM

Here is the email program of Append Only's chapters 2 and 4, designed this way. IMAP gives every message in a mailbox a UID that only grows, so UID order is arrival order, and the UID can be the sort key.

ACCESS PATTERN	OPERATION	KEY CONDITION
List an account's mailboxes	`Query`	`PK = ACCT#a`, `SK begins_with MBOX#`
Open a mailbox: its counts and newest 20	`Query`, reverse, limit 21	`PK = MBOX#a#INBOX`; the summary item, `SK = SUMMARY`, sorts after every `MSG#…`, so it comes first in reverse
Fetch one message	`GetItem`	`PK = MBOX#a#INBOX`, `SK = MSG#0000016720`
Page further back	`Query`, reverse, limit 20	same `PK`, `SK < MSG#0000016701`
Messages from a sender, newest first	`Query` on GSI1, reverse	`GSI1PK = FROM#a#sender`, sorted by `GSI1SK = date#uid`
A conversation	`Query` on GSI2	`GSI2PK = THREAD#t`, sorted by date
Unread in a mailbox	`Query` on GSI3, a sparse index	`GSI3PK = MBOX#a#INBOX`, written only while a message is unread

The mailbox, not the account, is the partition key of messages. That spreads one account's mail over as many partition keys as it has mailboxes. A 16,720-message INBOX is a few tens of megabytes, far below any partition limit, and email write rates are tiny. The design has no hot key until one mailbox gets thousands of messages a second.

> FOR YABIIR
> Every row of that table is a lookup or a prefix range over a sorted key, and that's all a sorted keydir needs to provide, once keys are tuples. The type prefixes are §14's Chapter 6. The indices are §14's Chapter 7, and the overloading and sparse-index tricks come with it for free: an index entry is a row the application, or the store, chooses to write. Whether a single-node book needs the partition-key half of this at all is §15's second question. The hot-key half doesn't apply: one node has no partitions to overload.

13 HOW 89 MILLION REQUESTS A SECOND FIT IN THREE REPLICAS

The abstract's number is easy to misread. "In 2021, during the 66-hour Amazon Prime Day shopping event, Amazon systems — including Alexa, the Amazon.com sites, and Amazon fulfillment centers, made trillions of API calls to DynamoDB, peaking at 89.2 million requests per second." Three things in that sentence are different from what it seems to say, and together they answer the question.

13.1 IT ISN'T THREE REPLICAS: IT'S THREE REPLICAS PER PARTITION

The replication factor is three **for each partition**, not for the service or for a table. §3: a table "is divided into multiple partitions", each with its own replication group. §4.3: "the latest generation of storage nodes hosts thousands of partition replicas." §6.1: DynamoDB runs "millions of Paxos groups in a Region". Each Paxos group is one partition's three replicas.

Replication and partitioning answer different questions. **Replication** answers "what happens when a disk dies," so it's set by durability and availability, and three, in three Availability Zones, is enough. **Partitioning** answers "how much can this table do," and it has no fixed number. A table gets more partitions as it grows in size or traffic (§4.4), and every partition brings its own three replicas with it. Throughput scales with the number of partitions. The number of replicas per partition doesn't change.

13.2 IT ISN'T ONE TABLE, AND THE REQUESTS DON'T TALK TO EACH OTHER

The 89.2 million requests a second are the sum over every table of every Amazon system that uses DynamoDB. The paper doesn't give a per-table breakdown. Whatever the split, almost every one of those requests is a single-item operation, a `GetItem` or `PutItem`, which touches exactly one partition and needs nothing from any other.

- **Routing** needs only the key-to-partition map, which every router caches and refreshes from MemDS without waiting (§6.6). Routers are stateless, so the router fleet is just made larger.
- **Admission** uses tokens the router already holds locally (§4.2). The central GAC service is consulted every few seconds, not on every request.
- **Authentication** uses cached IAM and KMS results (§6.5).
- **The storage node** does one lookup in one B-tree for a read, or for a write appends a log record and waits for one of its two peers.

No step of a single-item request involves a global lock, a global sequence or a global coordinator. The system is the same small system, a partition with three replicas, repeated millions of times, with a routing layer in front. Throughput adds up because nothing makes the copies wait for each other. Transactions are the exception, and the 2023 paper takes care to keep them off the path of ordinary gets and puts (§9).

13.3 THE ARITHMETIC

The paper gives no partition or node counts, so the following is an order-of-magnitude estimate from documented limits, not a measurement.

- A partition is limited to 3,000 read units a second [8]. If all 89.2 million requests were strongly consistent reads of small items, served by leaders alone, they'd need at least 89.2 M ÷ 3,000 ≈ 30,000 partitions running at their ceiling.
- Real traffic doesn't run every partition at its ceiling. With partitions averaging a tenth of their limit, about 300 requests a second, the same load needs about 300,000 partitions. That is well within the "millions of Paxos groups in a Region" of §6.1.
- Each partition has three replicas, so 300,000 partitions are 900,000 replicas. At "thousands of partition replicas" per storage node, that's a few hundred to a thousand nodes.
- A node hosting a few thousand replicas, each serving a few hundred requests a second, handles hundreds of thousands to a million small operations a second. That's a lot for one machine, but it's what NVMe storage and a fast network card are built for, and the paper's typical item is about a kilobyte.

The specific numbers are invented. The shape isn't: **a modest load per partition, multiplied by a very large number of independent partitions**. No single machine or replica group does anything remarkable.

13.4 READS GO TO ONE REPLICA, WRITES TO TWO

Replication also costs less than "three replicas" suggests.

- **An eventually consistent read is served by any one replica** (§3). The three copies are three servers that can each answer reads of the partition, so for those reads replication triples capacity instead of dividing it.

- **A strongly consistent read goes to the leader alone**, and needs no message to the other replicas while the leader's lease is valid. That's the point of the lease.
- **A write is acknowledged when two of the three have it in their logs**: the leader and the faster follower. The third catches up in the background, so a slow replica doesn't slow the write.

13.5 WHERE THE DIFFICULTY REALLY IS

Averages make it look easy, and §4 explains why it isn't. The total is easy to scale because it's a sum, but every individual key is limited to what one partition can do. A Prime Day deal that everyone clicks on is one item, and one item is one partition. The ten years of admission control in §4 are spent on that problem, not on the total: bursting, adaptive capacity, GAC, balancing and split-for-consumption. And the advice in §12 to give partition keys many evenly used values is how the application keeps its side of the deal.

> FOR YABIIR
> The lesson for a single-node store is about where scale comes from. yabiir's throughput is one machine's, and making it faster is a matter of the single node: group commit, the keydir, the cost of a `pread`. DynamoDB's throughput comes from never needing one node to be fast enough. A partitioned yabiir, if the book gets there, wouldn't go faster per node. It would go further by running many nodes, each owning a range of hashed keys and replicating its log to two others: Dynamo's design, with Bitcask under each node, as Riak ran it.

14 A LINE FOR THE BOOK

This section is a proposal, not a description of code. Nothing in it exists in yabiir yet, and every name in it is a placeholder.

The constraints were: the schema is given at open time, queries are trees, and migrations are out of scope. The papers above add one more, and a direction. The constraint: indices are few and explicit. The direction: build up to DynamoDB's data model on one node, then stop, or carry on into partitioning and replication in a later part.

14.1 CHAPTER 4: WRITES

Chapter 3 reads. The next chapter writes: `INSERT`, `DELETE`, and their conditions. An `INSERT` that fails when the key exists is `put` with the condition "absent". A `DELETE` that reports whether it deleted anything is `delete` with the condition "present". These are DynamoDB's condition expressions in their smallest form, and they need a read and a write under one lock. yabiir already serialises appends under one, so the check can go inside it. The tree grows a second statement, and the planner learns that a write has a plan too: a condition check and an append.

14.2 CHAPTER 5: COMPOSITE KEYS

The key becomes a tuple, and **the schema is the tuple's shape**, given as code when the store is opened:

SKETCH — A SCHEMA AT OPEN TIME (NOT CODE THAT EXISTS)

```
let schema = Schema::new("mail")
    .key("mailbox", Kind::Str)        // first part: what a Query fixes
    .key("received", Kind::U64)       // the rest: what a Query ranges over
    .key("uid", Kind::U32);
let mail = Table::open(dir, schema, Options::sorted())?;
```

The chapter's substance is the encoding. A tuple has to become bytes that sort the way the tuple does. Strings are terminated with `0x00`, and a `0x00` inside them is escaped as `0x00 0xFF`. Unsigned integers are written big-endian. Signed ones have their sign bit flipped. This is the FoundationDB tuple layer's encoding [11]. Once keys are encoded, Chapter 2's sorted keydir needs no change, and a new condition in the tree compiles to a `RangeScan` over a prefix:

SKETCH — THE TREE, EXTENDED (NOT CODE THAT EXISTS)

```
pub enum Condition {
    KeyEquals(Key),                              // get
    KeyBetween(Key, Key),                        // scan, Chapter 3
    Prefix { prefix: Key, low: Bound<Key>, high: Bound<Key> },  // DynamoDB's Query
}
pub struct Select {
    pub distinct: bool,
    pub fields: Fields,
    pub condition: Option<Condition>,
    pub order: Order,                            // Asc | Desc
    pub limit: Option<usize>,
}
```

The chapter can make the papers' central point with yabiir's own measurements. Open the same data hashed, and the planner has to answer a prefix condition with a `fold` and a filter. Open it sorted, and the planner walks one range. The cost difference is Chapter 2's B-tree footnote, now measured on a query someone would really make.

The engine work is a reverse scan with a limit: `scan` today is forward-only and returns a `Vec`. The email program's "newest twenty in INBOX" is a prefix `("INBOX")`, descending, limit 20, which reads twenty entries however large the mailbox is.

14.3 CHAPTER 6: ONE KEYSPACE, SEVERAL KINDS OF RECORD

A store with one key schema can still hold several kinds of record, if the first part of the key says which kind. A mailbox and its messages can share a prefix, so one range returns both:

KEY (TUPLE)	VALUE
`("INBOX")`	the mailbox: name, counts, UIDVALIDITY
`("INBOX", 1731400000, 16719)`	a message's headers
`("INBOX", 1731400420, 16720)`	a message's headers
`("Sent")`	the next mailbox

This is Spanner's interleaving, done in the key instead of declared in the schema, and it's the single-table design DynamoDB's documentation teaches. It replaces most of the joins a relational design would need. "The mailbox and its newest messages" isn't a join of two tables. It's one range of one keyspace.

Values become self-describing at this point: a small encoded map from attribute name to value, as DynamoDB's items are. `Fields` stops being three fixed choices and becomes a list of names. Since only the key has a schema, adding an attribute needs no migration, and old records simply lack it. That gives the book Appendix A's subject, the missing value, from a direction readers will recognise.

14.4 CHAPTER 7: AN INDEX IS A ROW YOU WRITE TOO

Chapter 4 in today's book is "Secondary Indices," and this is where it belongs: after composite keys, as a short chapter. An index on a message's sender is another record, with key `("by-sender", sender, received, uid)` and the message's primary key as its value, written with every write of the message. The chapter has three things to teach, each from the papers:

- **Atomicity.** The message and its index entry are written together or not at all. That needs the batch record of §9, and it is the one real storage-engine change in the proposal. Its API is DynamoDB's: a list of writes with conditions, submitted once.
- **Cost.** Every index adds a write to every write, which yabiir's benchmarks can measure: throughput with zero, one and three indices. That's SimpleDB's lesson, measured on the reader's own machine.
- **Choice.** An index is declared in the schema with the query it serves, and there are a few of them, not one per attribute. The planner chooses between the primary key and an index by rule, as Chapter 3's planner does. The cost-based planner stays in the "Rules and costs" box, where it is today.

14.5 CHAPTER 8: THE LOG IS THE TRUTH

The last chapter of the part comes back to the title. **Verify** rebuilds a keydir from the data files alone and compares it with the live one: DynamoDB's scrub, on one node. **Reading as of a time** rebuilds a keydir from the entries written up to that time: point-in-time restore. And the chapter ends on the tension the reconstruction exposes: merge is what makes a Bitcask small, and merge is what throws its history away.

14.6 WHAT THE LINE SKIPS, ON PURPOSE

SKIPPED	WHY THE PAPERS ALLOW IT
DDL and a catalog	The schema is code at open time. DynamoDB's only "DDL" is a key schema at creation, and it can't be altered.
Migrations	Only the key has a schema, and values describe themselves. Format changes use DynamoDB's read-then-write deployment, with merge as the rewrite.
General secondary indices	A few explicit indices, as rows. Spanner shipped without automatic ones, and F1 built its own.
Joins	Interleaved keys answer the common case, parent with children, as one range.
A cost-based planner	With few indices and rule-chosen access paths, rules are right. The "Rules and costs" box already says where cost-based planning begins.
MVCC and long transactions	DynamoDB's transactions are single-version and single-request by design.

14.7 AND THEN

What follows is partitioning and replication, and §4–§7 of the paper are the reading list for it. That means hashing the first key part to pick a partition, a log shipped to replicas, a quorum before acknowledgement, and leases. It also has the most natural possible starting point, because Bitcask was written to sit under one node of exactly that kind of system. That is a second part of the book, or a second book, and this report doesn't decide which.

15 OPEN QUESTIONS

1. **Is SQL text still the front end after Chapter 3?** The proposal keeps Chapter 3's parser and extends the tree. Prefix conditions, `ORDER BY ... DESC` and `LIMIT` have obvious SQL spellings, and the railroad diagrams could keep growing. Or the book could say that SQL was a way to name trees, and build trees directly from then on.
2. **Does the composite key replace the hashed keydir, or sit beside it?** DynamoDB hashes the first part and sorts the rest. A single-node store has no reason to hash for spreading load, so it could simply sort the whole tuple. Keeping a hashed option preserves Chapter 1's argument and costs a code path.
3. **Where does the batch record go in the format?** As a new entry type with a flag bit, as the tombstone flag is, it changes the data-file format. That makes it the first test of read-before-write format versioning, which is a chapter's worth of material in itself.
4. **How skewed should the book's benchmarks be?** Uniform keys flatter every design. A Zipfian default would match YCSB, and §4 of the paper.

PART II

LMDB

One file, one writer, no cache, no log. The 2011 paper that introduced it, the papers and pages around it on lmdb.tech, and the two independent studies that test its claims.

16 WHY READ LMDB AFTER DYNAMODB

Part I read a system at the largest scale there is: millions of replication groups, a fleet of routers, a decade of admission control. Part II reads the opposite. The Lightning Memory-Mapped Database is a C library of about 8,000 lines [12]. It runs inside one process on one machine and stores everything in one file. It has no cache, no write-ahead log, no background threads, and allows one writer at a time. Howard Chu wrote it for OpenLDAP, and his HPTS position paper says it plainly: "LMDB doesn't embody any radical new design concepts — all of its features existed separately in designs dating back to the 1960s. What's unique is the combination" [13].

It belongs in this report for three reasons.

- **It's yabiir's sibling.** Bitcask (2010) and LMDB (2011) were written at almost the same time, for the same reason: the authors had evaluated Berkeley DB and the other embedded stores of the day, and none did what they needed [4], [14]. Both answers never overwrite data a reader might be using. Bitcask appends records to a log. LMDB copies pages and reuses only the ones nobody can see. They're the two ends of Append Only's title.
- **It's what Chapter 2's B-tree looks like on disk.** yabiir's sorted keydir is a `BTreeMap` in memory, rebuilt on every open, and the whole keydir has to fit in RAM. That's Bitcask's one hard limit. LMDB's B+tree is the index and the data, on disk, and is ready the moment the file is mapped.
- **It answers Part I's unfinished questions on one node.** Part I's proposal needs atomic batches, several indices updated together, reverse range scans and a format-versioning policy. LMDB has all four, in a design small enough to read in an afternoon.

> SOURCES
> The main text is the LDAPCon 2011 paper that introduced the library, then called MDB [14], with its 2012 expansion for LinuxCon, Life After BerkeleyDB [15]. Around them are Chu's HPTS position paper [13], his 2015 SDC slides [12], the LMDB 1.0 documentation and benchmark pages on lmdb.tech [16], [17], [18], and two independent papers: the OSDI 2014 crash-consistency study that Chu cites [19], and the CIDR 2022 case against mmap [20]. Quotations are from the 2011 paper unless another source is cited.

17 THE PROBLEM: THREE CACHES AND TWO KINDS OF LOCK

The paper's §2 is a post-mortem of OpenLDAP's Berkeley DB backends, `back-bdb` (2002) and `back-hdb` (2003). They were, by Chu's account, the world's fastest directory servers since 2005, and their problems all came from one decision. BDB has its own cache, so the team hoped to read entries straight from it. "Early benchmark results showed that retrieving entries directly from the database on every query was too slow."

So they added an **entry cache** in the backend, and then a DN cache and an index cache, and the problems "grew from there":

Memory used three times the same data could be in the filesystem cache, the BDB cache and a backend cache at once, each with "different size and speed characteristics", each configured separately.

Two kinds of lock the backend caches used fast mutexes, but those had to be "dropped and exchanged for (much slower) database locks" whenever BDB was called, or deadlocks appeared that BDB's detector couldn't see. Even so, "deadlocks occur very frequently in routine operation of the backend."

Pathological behaviour when too small a cache smaller than the database churned through `malloc` and `free` until glibc's heap fragmented and the process grew, "defeating the purpose of setting a small cache size."

Zero effectiveness under scans queries whose results were larger than the cache evicted entries "before they ever get any chance of being re-used."

Logs to clean up BDB's write-ahead logs "grow continuously" and can be removed only after an expensive checkpoint. Its automatic log removal, if the system crashed while it was on, could leave a database that "could not be recovered successfully."

"From the advent of the back-bdb backend until the present time, the majority of development and debugging effort in these backends has all been devoted to backend cache management."

> THE SAME LESSON AS DYNAMODB'S, FROM THE OTHER END
> Part I's lesson four said not to let caches "hide the work that would be performed in their absence" (§7). OpenLDAP's experience is the single-machine version of that warning. Each cache was added to hide the cost of the layer below it, and the result was a system whose behaviour depended on which of three caches a request happened to hit. Both papers conclude that the cache was the problem. DynamoDB fixed it by making the cache do constant work. LMDB fixed it by removing the cache.

The paper's summary of the solution is two sentences long: "The problems with back-bdb and back-hdb can be summed up in two main areas: cache management, and lock management. The approach to a solution with back-mdb is simple — do no caching, and do no locking."

18 NO CACHING: THE SINGLE-LEVEL STORE

The idea is from Multics, in 1964: treat memory and disk as one address space, and let the operating system move pages between them. LMDB maps the whole database file into the process's address space with `mmap`. A read follows pointers through the map. If the page is in RAM it's used directly; if not, the CPU faults and the kernel reads it. There's no buffer pool, no eviction policy and no cache to size. The operating system's page cache is the only cache, and the database is in it once.

Three conditions make it work, and the paper states each:

- **A large address space.** The design was abandoned by the early 1990s because databases outgrew 32-bit addresses. With 64-bit processors that implement 48-bit virtual addresses, the limit is 47 bits, or 128 TB.
- **A unified buffer cache.** LMDB reads through the map but, by default, writes with ordinary `write()` calls, so the kernel has to show the same pages to both. Older systems kept the two separate. "Most modern operating systems now have filesystem and VM paging unified."
- **A read-only map.** The map is read-only by default. "Keeping the mapping read-only prevents stray writes from buggy application code from corrupting the database". In the 2011 slides, any stray write "will cause a SEGV, allowing immediate identification of software bugs." A writable map (`MDB_WRITEMAP`) came later, after the LinuxCon 2012 talk. It writes faster and gives up that protection [17].

The consequence that matters most for applications is **zero-copy reads**. `mdb_get` returns a pointer and a length into the map, not a copy. The documentation's instruction is "look but do not touch (or free() for that matter)," and the pointer is valid only until the transaction ends [16]. A read costs a tree descent and nothing else: no `malloc` , no `memcpy` , no system call if the pages are resident. That's why the benchmarks in §23 show LMDB's reads barely depending on value size.

FOR YABIIR
yabiir already follows half of this. It keeps no cache of values: `get` reads the value with `pread` (`read_exact_at`) and relies on the operating system's page cache, exactly as LMDB relies on it. The other half, zero copy, yabiir doesn't do: every value is copied into a `Vec<u8>`. A memory-mapped data file would remove that copy, and §24 is the argument for not doing it lightly.

19 COPY-ON-WRITE, AND TWO META PAGES

The data structure is a B+tree, the variant Comer's 1979 survey named and described: every key in the leaves, the levels above only an index to find them [21]. Its pages are never modified in place. To change a key, the writer copies the leaf, changes the copy, then copies each page on the path to the root so that it points at the new child. The result is a new root that shares every untouched page with the old one. Readers still holding the old root see the old tree, complete and unchanged.

LMDB didn't start that way. Its code began as Martin Hedenfalk's append-only B-tree from OpenBSD's `ldapd`, which wrote every new page at the end of the file, followed by a meta page pointing at the new root, and found the current root by searching backward from the end. The test that ended that design is worth quoting in full:

THE APPEND-ONLY B-TREE, MEASURED
"Initial testing with the append-only code proved that approach to be completely impractical. With a small test database and only a few hundred add/delete operations, the DB occupied 1027 pages but only 10 pages actually contained current data; over 99% of the space was wasted." A copy-on-write B-tree that only appends rewrites a whole root-to-leaf path for every change, so its file grows by several pages per write, and nothing ever reclaims them.

LMDB made two changes. First, it keeps freed pages in a **second B+tree**, the free list, and reuses them before growing the file (§20). Second, it replaced the search for the last meta page with two fixed ones. "In MDB there are two meta pages occupying page 0 and page 1 of the file. They are used alternately by transactions." Each records a transaction id and the roots of the data tree and the free-list tree. A commit writes the new pages and then overwrites the older meta page. "This is nothing more than standard double-buffering — any application opening the database uses the newer meta page, while a committer overwrites the older one."

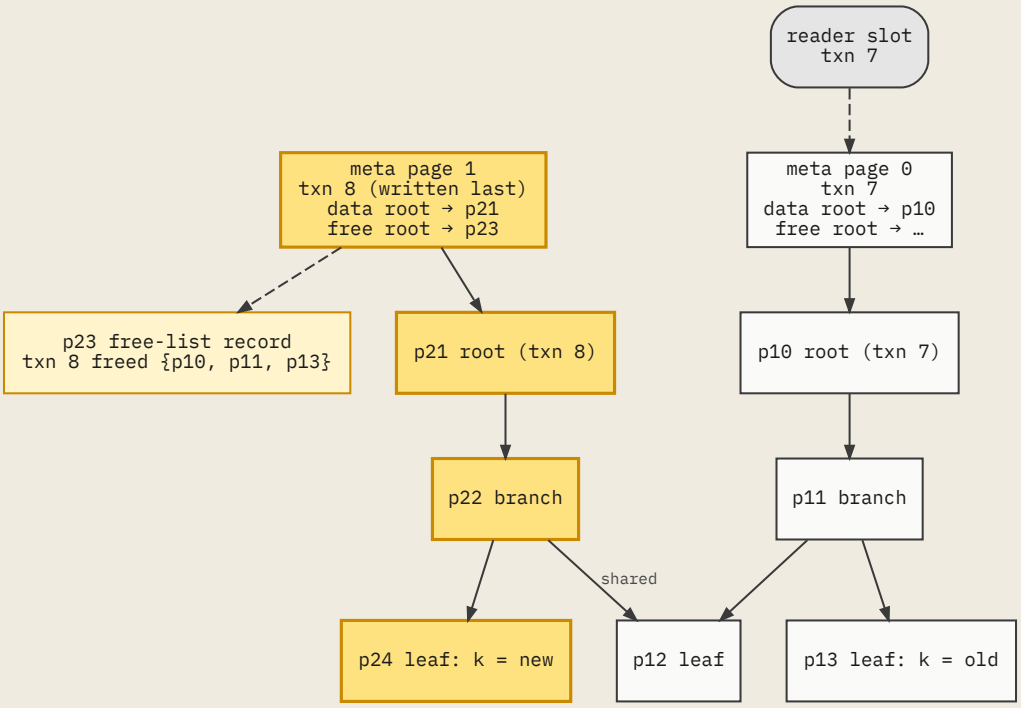


Figure 3: Transaction 8 changes one key. It copies the leaf (`p13` to `p24`) and the pages above it (`p11` , `p10`), records the three pages it replaced in the free list under its own id, and commits by writing meta page 1. Page `p12` is shared by both trees. A reader that began at transaction 7 still walks `p10` , `p11` and `p13` , which is why they can't be reused yet. Simplified: the free list is itself a copy-on-write B+tree.

That is the whole recovery story. There's nothing to replay, because the database on disk is always one of two complete trees, and the meta page with the higher valid transaction id says which. The 1.0 documentation puts it as "no special recovery procedures after a system crash" [16]. A crash during a commit leaves the new pages written but unreferenced, and the old meta page still points at the old tree.

19.1 "CRASH-PROOF," READ CLOSELY

Chu's later slides claim LMDB is "proven crash-proof in independent studies," and cite two OSDI 2014 papers [12]. One of them, by Zheng et al., doesn't mention LMDB in its text. The other, Pillai et al.'s study of crash consistency with the ALICE tool, does test it [19]. Among eleven applications, ALICE found 60 vulnerabilities. Seven applications could lose data, and both versions of LevelDB could return silently wrong results. LMDB had **one** vulnerability: "LMDB requires that the final pointer update (106 bytes) in the copy-on-write tree to be atomic." If the file system tears that meta-page write, the database opens only read-only. The authors note it was "discussed in a mailing list, though there is no available workaround."

So the honest summary is nearly crash-proof: one known assumption, that a 106-byte write within one sector isn't torn, among the best results in the study. The claim in the slides is stronger than its sources. The design deserves the reputation; the slide doesn't.

> FOR YABIIR
> Bitcask's crash story is the log's. Every entry has a CRC, recovery scans to the last valid one, and a torn tail is cut off. There's no root pointer to tear. LMDB's story is the root's: two copies of it and a transaction id. Both avoid replaying anything. The difference is where the atomic step is. In yabiir it's "this entry's CRC checks", and in LMDB it's "this meta page is newer". The book could set the two side by side in a chapter on crashes. ALICE itself, which records a workload's system calls and replays them in every order the file system allows, is a tool yabiir could run.

20 NO LOCKING: READERS, SLOTS AND THE FREE LIST

MVCC is what removes the locks: "Readers only ever see the snapshot of the database as it existed when a read transaction began, so they are fully isolated from writers." The hard part is the other half: when is

an old page safe to reuse? Reference counts would need locks. LMDB's answer is a table, and it is the most carefully engineered part of the design.

- **A reader table in shared memory.** The lock file holds one slot per active reader, recording its process id, thread id and the transaction id of its snapshot. Slots are laid out on separate cache lines to avoid false sharing.
- **One lock, once per thread.** The first time a thread opens a read transaction, it takes a mutex to claim a slot and remembers the slot in thread-local storage. After that, starting a read is storing a transaction id in the slot and ending it is zeroing it. "In normal operation, there is nothing that can block the operation of readers."
- **The free list is keyed by transaction.** When a writer copies a page, the original's id goes on an in-memory list. At commit the list is saved "as a single record in the free list DB along with the ID of the transaction for this commit."
- **Reuse when no reader is older.** Before taking pages from the free list, the writer compares the oldest free-list record's transaction id with every slot in the reader table. If the record is older than every reader, nothing can reach its pages, and they're reused. The scan takes no lock, "so readers cannot block writers."

The design's one failure mode follows directly. "The only consequence of a reader holding onto an old snapshot for a long time is that page reclaiming cannot be done; the writer will simply use newly allocated pages in the meantime." The 1.0 documentation lists it first among its caveats: "Avoid long-lived transactions. Read transactions prevent reuse of pages freed by newer write transactions, thus the database can grow quickly" [16]. A process that crashes while holding a slot leaves a stale reader, which has the same effect until `mdb_reader_check` clears it.

20.1 THE ARGUMENT AGAINST COMPACTION

The 2011 paper makes an argument that applies directly to yabiir. MVCC systems that always write to new pages "tend to grow without bound" and need compaction. Systems based on garbage collection "generally require twice as much disk space as the actual data occupies." And:

> CHU ON COMPACTION
> "In order to sustain a write rate of N operations/second, the I/O system must actually support >>2N operations/second, since the compaction task needs to run faster than the normal write task in order to catch up and actually complete its job, and the volume of data already written always exceeds the volume being written. If this over-provisioning of I/O resources cannot be guaranteed, then the typical solution to this problem is to deny updates while compaction is being performed."

Bitcask's merge is exactly that kind of compaction. It reads every live record, writes it to a new file, and deletes the old files. LMDB avoids it by reusing pages in place, at the cost of copy-on-write's own write amplification: every change rewrites a whole path from leaf to root. Neither choice is free. It's a choice between paying all the time (LMDB) and paying in bursts (Bitcask).

> FOR YABIIR
> This is a measurement the book could make, and should. How many bytes does merge write per byte of user data, at a given rate of overwrites? How much write throughput is left while a merge runs? yabiir's merge already rewrites files in batches (ISSS-TR-0501), so the numbers exist to be collected. Chu's ">>2N" is an argument, not a measurement, and the book can test it on a real compactor.

21 ONE WRITER

"For simplicity the MDB library allows only one writer at a time." A write transaction takes a mutex, which lives in the lock file so that it works across processes. That rule buys a great deal: writers "can never deadlock," as the documentation says [16]. No transaction can see another's uncommitted work. There is no conflict detection and no retry. It costs exactly what you would expect. The paper's own modify test shows `back-mdb` at 6,132 modifies per second against `back-hdb`'s 20,441, though each modify took less

time (1.29 ms against 1.56 ms). “Back-mdb actually completes modifies quickly, but because MDB enforces single-writer behavior, it does not accept as many writes per second.”

Two features make the single writer cheaper than it sounds:

- **Batching is free.** A write transaction can contain any number of puts and deletes, across any number of named databases (§22), and commits them all with one meta-page write. The more work per transaction, the less each operation pays for the commit’s `fsync`.
- **Bulk loads append.** `MDB_APPEND` tells LMDB that keys arrive in order, so it can fill pages sequentially “at the full sequential write speed of the underlying storage system” [15].

Durability is configurable. `MDB_NOSYNC` and `MDB_NOMETASYNC` skip some or all of the commit’s `fsync` calls, trading the latest transactions for speed. The documentation calls it “(temporarily) reduce robustness to gain even more speed” [16]. That is the same choice yabiir makes with `sync_on_put`.

> FOR YABIIR
> yabiir also has one writer, and since this report was first written it has been made strict: an append and the keydir update that publishes it are one step under the writer’s lock (`crates/yabiir/src/writer.rs`), so the keydir changes in the order of the log. What it lacks is the transaction around them. In LMDB, atomicity comes for free from the meta-page swap, because a transaction’s pages become visible all at once or not at all. In a log, atomicity has to be written into the log, as the batch record of Part I’s §9. The two designs reach the same API (several writes, one commit) by opposite mechanisms.

22 A DATA MODEL IN A LIBRARY

LMDB’s API is small, but it has the pieces Part I’s proposal needs, and back-mdb uses them in the way the proposal suggests.

Named databases one environment (one file) holds any number of named B+trees, each opened by name, all sharing one transaction. “The back-mdb indexing code uses individual databases for each attribute index.” Every attribute index is its own tree, and **an entry and all its index entries change in one transaction**, atomically, with no extra machinery. It’s the “index is a row you write too” of Part I’s Chapter 7, except the rows go into separate trees rather than prefixes of one keyspace.

Sorted duplicates with `MDB_DUPSORT`, one key holds a sorted set of values. That’s the shape of an index posting list (attribute value → the ids of every entry that has it). It’s implemented as a sub-tree per key: “a data item in a Btree to be treated as the root node of another Btree.”

Key order as data custom comparison functions are a hazard, because a tool that opens the database without them sees keys in the wrong order and corrupts it. LMDB instead stores flags in the database header (`MDB_INTEGERKEY` for native integers, `MDB_REVERSEKEY` for reversed byte order), “so that any MDB application will sort correctly without any special effort” [15]. It’s the argument for Part I’s order-preserving tuple encoding: if the bytes sort correctly, no code needs to know how.

Cursors `MDB_SET_RANGE` positions at “the first key greater than or equal to” a given key. With `MDB_NEXT`, `MDB_PREV`, `MDB_LAST` and their duplicate-aware variants, it gives forward and reverse range scans, prefix scans, and DynamoDB’s floor and ceiling lookups from Part I’s §7.

Nested transactions a write transaction can open a child write transaction, which commits into its parent or aborts alone. 1.0 adds read-only children of a write transaction [16].

None of this is SQL, and none of it is a schema. It is a sorted, transactional, multi-tree key-value store, and an LDAP server was built on it in five days: “the MDB library code was feature-complete August 31, 2011 at which point coding of back-mdb began. Back-mdb was feature-complete 5 days later” [15]. That’s close evidence for Part I’s thesis that a small, sorted, transactional key-value core is enough for a real application’s data model.

23 THE RESULTS, READ CLOSELY

23.1 THE 2011 PAPER

All tests used a 5-million-entry directory on the authors' machine, with the database fully cached in RAM. Under those conditions, the numbers are dramatic:

TEST	BACK-HDB (BDB)	BACK-MDB (LMDB)
Load 5 M entries (`slapadd -q`), wall clock	66 m 10 s	29 m 33 s
Load, user CPU	115 m 52 s (multi-threaded)	22 m 21 s
Scan everything, first run	4 m 15 s	14.7 s
Scan everything, second run	16.2 s	10.8 s
Process size / database size	26 GB / 15.6 GB	10 GB / 12.8 GB
Same scan ×16 concurrently, average	3 m 04 s	12.0 s
Random searches per second (SLAMD)	67,456	119,255
Modifies per second (SLAMD)	20,441	6,132
Searches + modifies per second, together	40,629 + 12,321	85,919 + 2,845

Three details are worth more than the headline ratios.

- **Concurrency is flat.** LMDB's scan took 10.8 s with 2 concurrent clients and 12.0 s with 16. `back-hdb` went from 24.6 s to 3 m 04 s, and the stock Debian build of BDB, with its default spinlocks, reached 17 m 15 s. Readers that take no locks scale with the number of cores until the cores run out.
- **A scan ran at disk-copy speed.** Copying the database file to `/dev/null` with `dd` took 10.2 s. The full ldapsearch, decoding and filtering every entry, took 10.8 s, "only 6% slower than a raw copy operation." That is what having no cache layer means in practice.
- **Writes are the price.** Under a combined load, `back-mdb` 's modify rate fell to 2,845 a second, less than a quarter of `back-hdb` 's. The paper says so and moves on: "it is not perceived as a critical problem for now." LDAP directories are read-mostly. The design fits the workload, and the paper never claims it fits every workload.

23.2 THE MICROBENCHMARKS

The 2012 paper adapted Google's LevelDB benchmark (`db_bench`) to LMDB, BDB, SQLite and Kyoto Cabinet [15], and the lmdb.tech pages extend it [17], [18]. On fully cached reads of 1 million records (16-byte keys, 100-byte values), LMDB did 14.5 million sequential and 769,000 random reads a second, against LevelDB's 4.5 million and 187,000. With large values its sequential reads reached 33 million a second against LevelDB's 195,000, because a zero-copy read costs the same whatever the value's size.[2]

The same pages are candid about the other side.

- **LevelDB wins on writes.** "With a few exceptions, Google's LevelDB handily outperforms all of the other databases" on asynchronous writes [15].
- **Small values are expensive.** "At small value sizes the overhead of the Copy-on-Write Btree approach in LMDB is very high, much higher than for the LSM-based engines. At larger value sizes the COW overhead is proportionally smaller, and the LSM write amplification starts to dominate" [18].
- **Space has a cost too.** There's an 8-byte header per record, which on the 2014 test's 4 billion records was 32 GB of extra disk [18].
- **They're the authors' own benchmarks.** The pages document their setup carefully: CPU frequency pinned, single-user mode, median of three runs, a fix for LevelDB's duplicate keys in random writes. Every number, though, comes from the people who wrote LMDB.

[2] The paper describes this test as "a database with 100,000 byte keys, and 1,000 records." LMDB 0.9's maximum key size was hardcoded at 512 bytes [16], so the 100,000 bytes must be values; the sentence after it, about "the size of the records being returned," agrees.

23.3 SIZE

The footprint figures are not marketing. The 2015 slides list lines of code for each engine's benchmark build: LMDB 7,955, LevelDB 39,758, RocksDB 81,169, WiredTiger 91,410, TokuDB 227,698, and Berkeley DB 1,746,106 [12]. Object code was under 32 KB, small enough, Chu argues, that "the core of the library fits entirely within a CPU's Level 1 cache" [15].

24 THE CASE AGAINST MMAP

Crotty, Leis and Pavlo's CIDR 2022 paper is the strongest published argument against LMDB's central choice. It names LMDB as the system that "relies on mmap exclusively" [20], and it concludes that a DBMS should consider mmap only when the working set fits in memory and the workload is read-only, "otherwise, never." Its four problems, read against LMDB's design:

PROBLEM	THE PAPER'S ARGUMENT	AGAINST LMDB
1. Transactional safety	The OS may write a dirty mapped page to disk at any time, even a locked one, so changes can reach disk before their transaction commits.	Doesn't apply as described. With the default read-only map there are no dirty mapped pages, since writes go through `write()`. With `MDB_WRITEMAP`, a dirty page is always a new copy that no committed meta page points to, so writing it early is harmless. The paper describes LMDB as flushing a shadow copy with `msync` and swapping a pointer. LMDB's shadow is per page, in one file.
2. I/O stalls	Any page access can fault and block, with no asynchronous reads and only hints (`madvise`) to steer prefetching.	Applies. A read on a database larger than memory can stall on any page, and LMDB has no way to issue reads ahead.
3. Error handling	Checksums would have to be checked on every access; pointer bugs persist silently; I/O errors arrive as `SIGBUS` anywhere.	Partly applies. The read-only map stops pointer bugs from reaching disk. 1.0 adds page-level checksums [16]. `SIGBUS` on an I/O error is inherent to mmap.
4. Performance	Page eviction is limited by TLB shootdowns, a single `kswapd` thread, and page table contention. Measured: random reads fell to about half of `pread`'s on a 2 TB range with 100 GB of page cache, after dropping to near zero for five seconds.	Applies to larger-than-memory random reads on fast NVMe arrays, which is not the workload LMDB was designed or benchmarked for.

The two sides agree more than they appear to. The paper's "maybe use mmap" case is a working set that fits in memory and a read-mostly workload. That's LMDB's case exactly: LDAP directories, fully cached, read-heavy, which is what every 2011 benchmark measured. The real disagreement is about how often a database stays in that case. A clear reading of both papers is that LMDB is the best design for the workload it chose, and that its benchmarks don't tell you what happens outside it.

> **FOR YABIIR**
> yabiir reads with `pread`, and that's the right default for a book. It avoids every problem in the table, it reads like ordinary file I/O, and it keeps I/O errors as `Result`s, not signals. A memory-mapped read path could be a measured experiment, but not the main design. Its benefit, zero copy, matters most for large values, and its risks matter most for databases larger than memory.

25 LMDB 1.0

The documentation on lmdb.tech is for 1.0, and its upgrade page lists what changed since 0.9 [16]. Two items connect to Part I.

A format change with no compatibility path. "The on-disk file format has changed in LMDB 1.0 and versions 0.9 and 1.0 are mutually incompatible. You must use v0.9 `mdb_dump` to export your old DBs, and import with v1.0 `mdb_load`. … There is no support in LMDB 1.0 for operating directly on v0.9 DB files. Including such support would only bloat the library so it will not be done." That is the opposite of DynamoDB's read-then-write deployments (§7): a library can make the user migrate, offline, once a decade. A service with millions of tables can't.

Every page records the transaction that wrote it. 1.0 adds the transaction id to every page header. That allows **incremental backup**, which copies only the pages newer than a given transaction. It's DynamoDB's log-based backup (§6) without a log: the B+tree's pages carry their own history marks.

The rest:

- page-level checksums and encryption, with an optional key for keyed hashes;
- databases on raw block devices, where "writes are always synchronous, so there is no need to do any fsyncs";
- two-phase commit, where "preparing a transaction just means writing all of the transaction's data except for the final metapage update";
- page sizes up to 64 KB;
- key size limits that follow the page size.

Each one fits the design without changing it. Two-phase commit in particular falls out of the meta-page swap almost for free.

26 READING ACROSS THE TWO PARTS

	DYNAMODB	LMDB	YABIIR (BITCASK)
Unit	a partition: 3 replicas	one file, one process tree	one directory of logs
Structure	WAL + B-tree per replica	copy-on-write B+tree, no log	log only; keydir in RAM
Writers	one leader per partition	one per file	one append lock
Readers	any replica, or the leader	lock-free snapshots	concurrent, keydir snapshot
Atomic unit	one item; transactions in one request	a transaction, any size	one record
Indices	few GSIs and LSIs, async or local	named trees in the same transaction	none
Crash recovery	replay WAL; log replicas	none: pick the newer meta page	scan logs or read hints; cut torn tail
Space reclaimed by	B-tree in place; logs archived	free list, when no reader is older	merge (compaction)
Caches	constant-work metadata cache	none: the page cache only	none: the page cache only
Format change	read first, write later	dump and load	—

The table shows three things.

- **The same lesson about caches.** DynamoDB made its cache do constant work. LMDB and Bitcask have no cache at all and leave caching to the operating system. All three treat a cache that changes how much work the system does as a hazard.

- **Never overwrite what a reader can see.** DynamoDB's storage nodes keep a log in front of the B-tree. LMDB copies pages and reuses only invisible ones. Bitcask appends. Each one is a way of being append only, and the book's title covers all three.
- **Atomicity is cheapest where the structure has a single root.** LMDB gets transactions of any size from one meta-page write. Bitcask has no root, only a log, so atomicity has to be put into the log as a record that spans several writes. DynamoDB, with no root across partitions, needs a coordinator and a ledger.

27 FOR YABIIR AND THE BOOK

These notes are proposals, in the spirit of Part I's §14.

1. **A chapter on crashes, two ways.** Put Bitcask's recovery (CRC, scan, cut the tail) beside LMDB's (two meta pages, take the newer), and run both under a crash simulator. Pillai et al.'s result, one 106-byte assumption for LMDB, is the bar.
2. **Measure merge the way Chu argues about it.** Bytes written per byte of user data, and write throughput during a merge, at a range of overwrite rates. That turns ">>2N" from a quotation into a chart.
3. **Snapshots as an API.** yabiir's `fold` and `scan` already pin a point-in-time snapshot of the keys they visit. LMDB shows the API that grows from that: a read transaction that sees one consistent version across many calls. In a Bitcask that's a pinned keydir snapshot plus pinned files, which merge has to respect, much as LMDB's free list respects the reader table.
4. **The batch record is the transaction.** Part I's batch record and LMDB's write transaction are the same API. Implementing the first lets the book show the second's semantics without its data structure.
5. **A keydir that doesn't fit in memory.** Bitcask's known limit is that every key must fit in RAM. LMDB is the standard answer, an on-disk B+tree, and it is also Chapter 2's B-tree made real. A later part of the book could build a small copy-on-write B+tree over the same `Bitcask` trait, starting where LMDB started: an append-only tree, measured until 99% of the file is garbage.
6. **Keep `pread`.** The case for mmap is zero copy. The case against it is in a peer-reviewed paper. A book should take the safe default and measure the alternative.

28 OPEN QUESTIONS, PART II

1. **Is a second engine in scope?** A copy-on-write B+tree is a different book's worth of code. The smaller step is to read LMDB's design in a chapter, and implement only the snapshot API and the batch record in yabiir.
2. **Should yabiir's read path offer zero copy?** A borrowed-value API (`get_ref`, valid while a read snapshot lives) is possible with `pread` into a reusable buffer, without mmap. It would show the API cost of zero copy without the risks.
3. **Where does the book stand on the one-writer rule?** yabiir now has it as a documented rule (ROADMAP §3, and the module documentation of `writer.rs`): one writer, one total order of writes, and sharding the keydir or the active file is out of scope. Saying what that gives up, as LMDB's paper does with its modify benchmark, is still a paragraph the book owes. Arguing against the rule needs a benchmark.

28 REFERENCES

[1] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in 2022 USENIX Annual Technical Conference (USENIX ATC 22), 2022, pp. 1037–1048.

[2] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-Value Store," in Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07), 2007, pp. 205–220.

[3] J. Idziorek et al., "Distributed Transactions at Scale in Amazon DynamoDB," in 2023 USENIX Annual Technical Conference (USENIX ATC 23), 2023, pp. 705–717.

[4] J. Sheehy and D. Smith, "Bitcask: A Log-Structured Hash Table for Fast Key/Value Data," technical report, 2010.

[5] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff, "How Amazon Web Services Uses Formal Methods," Communications of the ACM, vol. 58, no. 4, pp. 66–73, 2015.

[6] P. Huang et al., "Gray Failure: The Achilles' Heel of Cloud-Scale Systems," in Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS '17), 2017, pp. 150–155.

[7] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking Cloud Serving Systems with YCSB," in Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC '10), 2010, pp. 143–154.

[8] "Amazon DynamoDB Developer Guide." [Online]. Available: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/

[9] J. C. Corbett et al., "Spanner: Google's Globally-Distributed Database," in 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12), 2012, pp. 251–264.

[10] A. DeBrie, The DynamoDB Book. self-published, 2020.

[11] "Data Modeling: Tuples." [Online]. Available: https://apple.github.io/foundationdb/data-modeling.html

[12] H. Chu, "LMDB: The Lightning Memory-Mapped Database." [Online]. Available: http://www.lmdb.tech/media/20150921-SDC-LMDB.pdf

[13] H. Chu, "HPTS Position Paper." [Online]. Available: http://www.lmdb.tech/media/HPTS-Position.pdf

[14] H. Chu, "MDB: A Memory-Mapped Database and Backend for OpenLDAP," in LDAPCon 2011, 2011. [Online]. Available: http://www.lmdb.tech/media/20111010LDAPCon%20MDB.pdf

[15] H. Chu, "Life After BerkeleyDB: OpenLDAP's Memory-Mapped Database," in LinuxCon North America, 2012. [Online]. Available: http://www.lmdb.tech/media/20120829-LinuxCon-MDB-txt.pdf

[16] "LMDB 1.0 Documentation: Introduction, Getting Started, Upgrading From Release 0.9." [Online]. Available: http://www.lmdb.tech/doc/

[17] "Database Microbenchmarks." [Online]. Available: http://www.lmdb.tech/bench/microbench/

[18] "On-Disk Microbenchmark." [Online]. Available: http://www.lmdb.tech/bench/ondisk/

[19] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications," in 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14), 2014, pp. 433–448.

[20] A. Crotty, V. Leis, and A. Pavlo, "Are You Sure You Want to Use MMAP in Your Database Management System?," in 12th Conference on Innovative Data Systems Research (CIDR '22), 2022.

[21] D. Comer, "The Ubiquitous B-Tree," ACM Computing Surveys, vol. 11, no. 2, pp. 121–137, 1979.